CSS311 – Parallel and Distributed Computing

Lab 2: False Sharing, SISD/SIMD, and OpenMP Parallelism

NAME : VIINEETH MN
ROLLNO : 2024BCS0178

Question 1: OpenMP – Pi Estimation

1. Write an OpenMP program with C / C++ that estimates the value of pi (π) using a following numerical integration formula called rectangle rule.

$$Area = \int_0^1 \frac{4}{1+x^2} dx = \pi$$

The following components are to be shown.

(a) Write the sequential version program to estimate the value of pi (π). Test the result with classical integration value.

PADS > Lab2 > q1a.cpp > ...

```cpp
// CSS311 - Lab 2 - Question 1(a): Sequential Pi Estimation (Rectangle Rule)

#include <iostream>
#include <cmath>
#include <chrono>
using namespace std;
using namespace std::chrono;

int main() {
    long long n = 1000000;          // >= 10^6 intervals as required
    double step = 1.0 / (double)n;

    auto t0 = high_resolution_clock::now();

    double sum = 0.0;
    for (long long i = 0; i < n; i++) {
        double x = (i + 0.5) * step;   // midpoint of interval i
        sum += 4.0 / (1.0 + x * x);
    }
    double pi = sum * step;

    auto t1 = high_resolution_clock::now();
    double elapsed = duration<double>(t1 - t0).count();

    cout << "===== Sequential Version =====\n";
    cout << "Computed pi = " << pi << "\n";
    cout << "Classical pi (M_PI) = " << M_PI << "\n";
    cout << "Absolute error = " << fabs(pi - M_PI) << "\n";
    cout << "Time = " << elapsed << " sec\n";

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[vini@asus Lab2]$ ./q1a
===== Sequential Version =====
Computed pi = 3.14159
Classical pi (M_PI) = 3.14159
Absolute error = 2.88658e-14
Time = 0.00656053 sec
[vini@asus Lab2]$
```

Sc - program and output

(b) Write the parallel version program to estimate the same. Test the result with classical

integration value and by (a). It includes number of threads involved and the area calculated by
which thread number.

PADS > Lab2 > q1b.cpp > ...

```cpp
// CSS311 - Lab 2 - Question 1(b): Parallel Pi Estimation

#include <iostream>
#include <omp.h>
#include <cmath>
using namespace std;

int main() {
    long long n = 1000000;          // >= 10^6 intervals as required
    double step = 1.0 / (double)n;
    int nthreads = 4;
    omp_set_num_threads(nthreads);

    double sum_par = 0.0;   // shared variable updated by multiple threads
    double t0 = omp_get_wtime();

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        double local_sum = 0.0;

        #pragma omp for
        for (long long i = 0; i < n; i++) {
            double x = (i + 0.5) * step;
            local_sum += 4.0 / (1.0 + x * x);
        }

        // show number of threads involved and area calculated by each thread
        cout << "Thread " << tid << " -> partial area = " << local_sum * step << "\n";

        sum_par += local_sum;   // shared accumulation (race condition, see part c)
    }

    double pi_par = sum_par * step;
    double t1 = omp_get_wtime();

    cout << "\n===== Parallel Version =====\n";
    cout << "Number of threads = " << nthreads << "\n";
    cout << "Computed pi = " << pi_par << "\n";
    double pi_true = acos(-1.0);
    cout << "Classical pi = " << pi_true << "\n";
    cout << "Absolute error = " << fabs(pi_par - pi_true) << "\n";
    cout << "Time = " << (t1 - t0) << " sec\n";

    return 0;
}
```

Sc - program

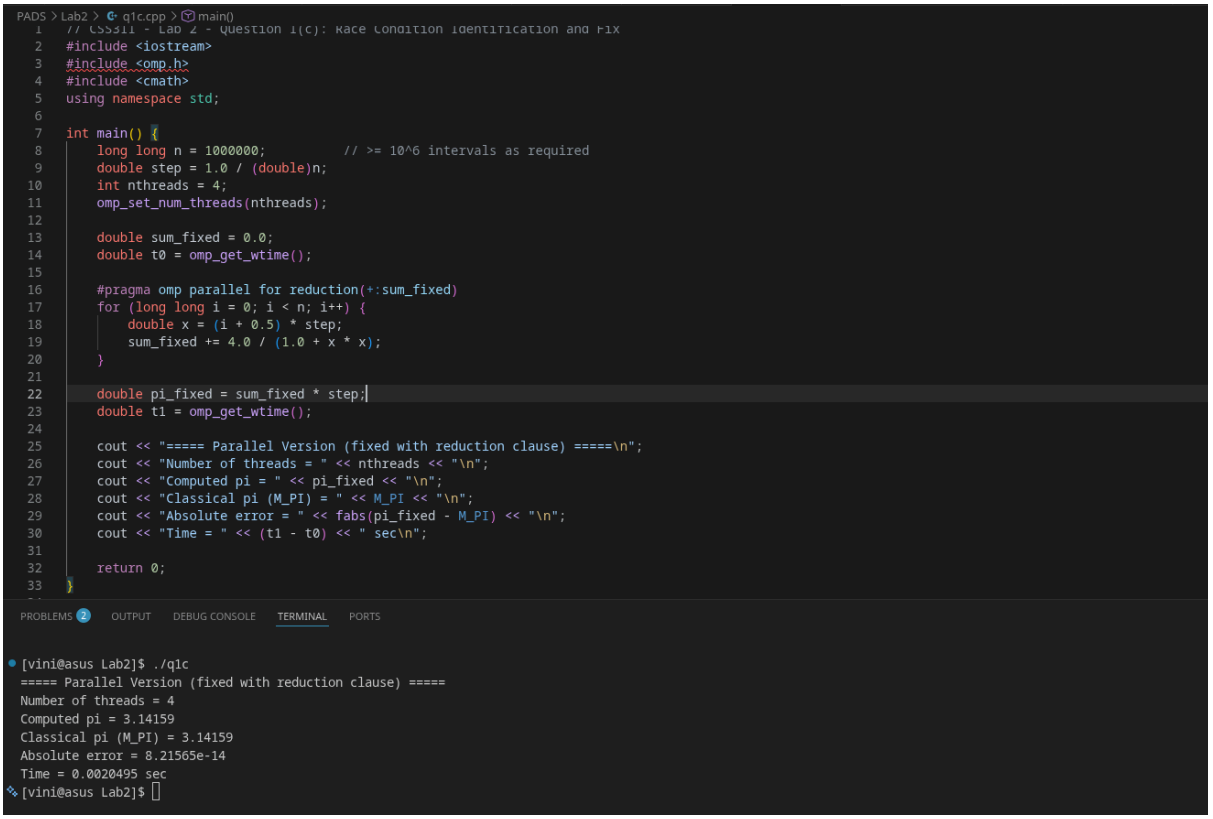
```
[vini@asus Lab2]$ ./q1b
Thread Thread 21 -> partial area =  -> partial area = 0.719414
Thread 0.8746763 -> partial area =
0.567588
Thread 0 -> partial area = 0.979915

===== Parallel Version =====
Number of threads = 4
Computed pi = 3.14159
Classical pi = 3.14159
Absolute error = 8.21565e-14
Time = 0.00361011 sec
[vini@asus Lab2]$
```

Sc - output

(c) Identify the line of statement which leads the race condition. Race condition occurs when the

multiple threads accessing a shared variable. If it exists, how will you handle this problem? Use appropriate OpenMP clause and find the solution. Test the result with classical integration
value and by (a) and (b).



Sc - program and output

Question 2: False Sharing
Investigate the impact of false sharing on parallel performance using OpenMP.

(a) Create an OpenMP program where each thread repeatedly updates its own counter stored in a
contiguous integer array (i.e., all counters packed together in memory). Measure and display the execution time.

PADS > Lab2 > q2a.cpp > main()

```cpp
int main() {
    int nthreads = 4;
    omp_set_num_threads(nthreads);

    // All counters packed together in one contiguous int array.
    // Threads writing to neighbouring array elements repeatedly invalidate
    // the same CPU cache line -> false sharing.
    vector<int> counters(nthreads, 0);

    double t0 = omp_get_wtime();
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        for (long long i = 0; i < ITER; i++) {
            counters[tid]++;
        }
    }
    double t1 = omp_get_wtime();

    cout << "===== False Sharing Version (contiguous array) =====\n";
    cout << "Number of threads = " << nthreads << "\n";
    for (int i = 0; i < nthreads; i++)
        cout << "counters[" << i << "] = " << counters[i] << "\n";
    cout << "Time = " << (t1 - t0) << " sec\n";

    return 0;
}
```

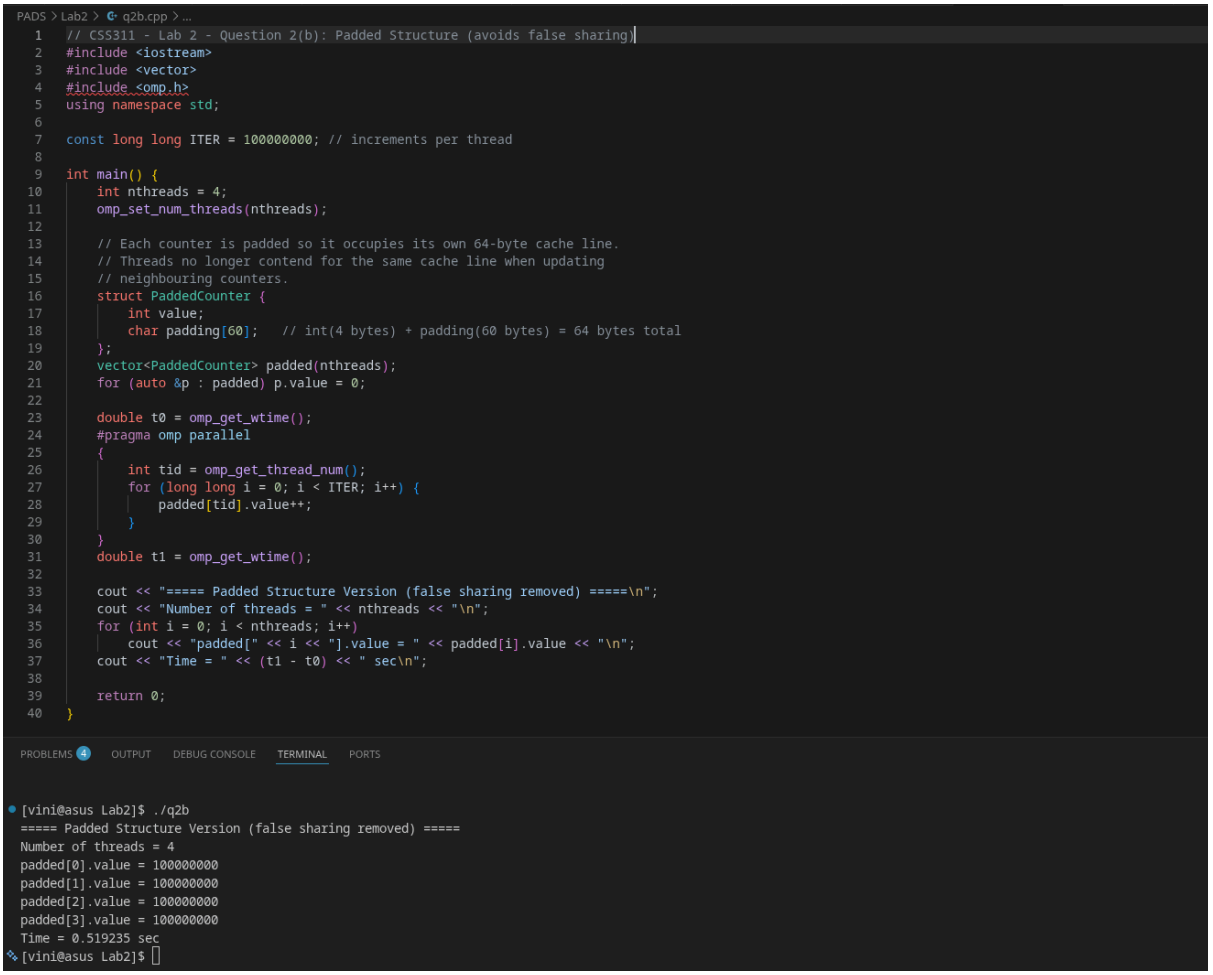
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[vini@asus Lab2]$ ./q1c
===== Parallel Version (fixed with reduction clause) =====
Number of threads = 4
Computed pi = 3.14159
Classical pi (M_PI) = 3.14159
Absolute error = 8.21565e-14
Time = 0.0020495 sec
[vini@asus Lab2]$ g++ q2a.cpp -o q2a -fopenmp
[vini@asus Lab2]$ ./q2a
===== False Sharing Version (contiguous array) =====
Number of threads = 4
counters[0] = 100000000
counters[1] = 100000000
counters[2] = 100000000
counters[3] = 100000000
Time = 2.64064 sec
[vini@asus Lab2]$
```

Sc - program and output

(b) Modify the program using a padded structure so that each thread's counter occupies a separate

cache line (typically 64 bytes). Measure and display the execution time.



```
PADS > Lab2 > q2b.cpp > ...
// CSS311 - Lab 2 - Question 2(b): Padded Structure (avoids false sharing)
#include <iostream>
#include <vector>
#include <omp.h>
using namespace std;

const long long ITER = 100000000; // increments per thread

int main() {
    int nthreads = 4;
    omp_set_num_threads(nthreads);

    // Each counter is padded so it occupies its own 64-byte cache line.
    // Threads no longer contend for the same cache line when updating
    // neighbouring counters.
    struct PaddedCounter {
        int value;
        char padding[60];   // int(4 bytes) + padding(60 bytes) = 64 bytes total
    };
    vector<PaddedCounter> padded(nthreads);
    for (auto &p : padded) p.value = 0;

    double t0 = omp_get_wtime();
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        for (long long i = 0; i < ITER; i++) {
            padded[tid].value++;
        }
    }
    double t1 = omp_get_wtime();

    cout << "===== Padded Structure Version (false sharing removed) =====\n";
    cout << "Number of threads = " << nthreads << "\n";
    for (int i = 0; i < nthreads; i++)
        cout << "padded[" << i << "].value = " << padded[i].value << "\n";
    cout << "Time = " << (t1 - t0) << " sec\n";

    return 0;
}

PROBLEMS 4  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

[vini@asus Lab2]$ ./q2b
===== Padded Structure Version (false sharing removed) =====
Number of threads = 4
padded[0].value = 100000000
padded[1].value = 100000000
padded[2].value = 100000000
padded[3].value = 100000000
Time = 0.519235 sec
[vini@asus Lab2]$
```

Sc - program and output

(c) Implement a third version using the OpenMP reduction clause. Compare all three versions
namely false sharing, padded, and reduction in terms of execution time and explain the observed
performance differences.

PADS > Lab2 > q2c.cpp > ...

```cpp
// CSS311 - Lab 2 - Question 2(c): OpenMP Reduction Clause + Comparison
#include <iostream>
#include <vector>
#include <omp.h>
using namespace std;

const long long ITER = 100000000; // increments per thread

int main() {
    int nthreads = 4;
    omp_set_num_threads(nthreads);

    // (a) False sharing version (for comparison)
    vector<int> counters(nthreads, 0);
    double t0 = omp_get_wtime();
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        for (long long i = 0; i < ITER; i++) counters[tid]++;
    }
    double t1 = omp_get_wtime();
    double time_a = t1 - t0;

    // (b) Padded version (for comparison)
    struct PaddedCounter {
        int value;
        char padding[60];
    };
    vector<PaddedCounter> padded(nthreads);
    for (auto &p : padded) p.value = 0;

    double t2 = omp_get_wtime();
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        for (long long i = 0; i < ITER; i++) padded[tid].value++;
    }
    double t3 = omp_get_wtime();
    double time_b = t3 - t2;

    // (c) OpenMP reduction version

    long long total = 0;
    double t4 = omp_get_wtime();
    #pragma omp parallel reduction(+:total)
    {
        long long local = 0;
        for (long long i = 0; i < ITER; i++) local++;
        total += local;
    }
    double t5 = omp_get_wtime();
    double time_c = t5 - t4;

```

PADS > Lab2 > q2c.cpp > main()

```cpp
int main() {

    cout << "(c) OpenMP Reduction Clause Version \n";
    cout << "Number of threads = " << nthreads << "\n";
    cout << "Total = " << total << "\n";
    cout << "Time = " << time_c << " sec\n\n";

    cout << "Comparison of all three versions \n";
    cout << "(a) False sharing (contiguous array) time = " << time_a << " sec\n";
    cout << "(b) Padded structure time                 = " << time_b << " sec\n";
    cout << "(c) Reduction clause time                  = " << time_c << " sec\n\n";

    return 0;
}
```

Sc - program

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[vini@asus Lab2]$ ./q2c
(c) OpenMP Reduction Clause Version
Number of threads = 4
Total = 400000000
Time = 0.0735249 sec

Comparison of all three versions
(a) False sharing (contiguous array) time = 2.76376 sec
(b) Padded structure time                 = 0.481076 sec
(c) Reduction clause time                  = 0.0735249 sec

[vini@asus Lab2]$
```

Sc- output

Explanation of the performance differences:

Version (a) is the slowest because all counters are stored next to each other in the same array, so they end up sharing one or two CPU cache lines. Every time a thread increments its own counter, it invalidates that cache line for all the other cores, even though each thread is only touching its own logical variable. This forces the cores to constantly re-synchronize the cache line between them, an effect known as cache line "ping-ponging," which adds a lot of overhead.

Version (b) is significantly faster than (a) because each counter is padded out to occupy a full 64-byte cache line on its own. Since no two threads' counters share a cache line anymore, the cache-coherence traffic disappears entirely and each thread can update its counter without interfering with the others.

Version (c) is the fastest and most efficient of the three because each thread accumulates its increments in a private local variable that never touches shared memory during the loop. OpenMP's reduction clause then combines all the threads' private results into the final total safely and only once, after the loop finishes, avoiding both false sharing and any need for explicit synchronization.